Bot Traxion

Guía de estudio para la exposición oral

Automatización del alta de puntos de venta en la plataforma Traxion / Pago24 (TRASA): de la base de Pago24 Red al "Monto crédito = 1", de punta a punta.

Qué vas a poder explicar después de estudiar esta guía
1. Qué problema resuelve el bot y para quién.
2. La arquitectura completa: app → Boss → Worker → Traxion, y la base de datos.
3. Cómo se automatiza un sitio web difícil (Seaside + AJAX) sin depender de tiempos fijos.
4. Cómo se diseñó la base para no hacer full scans y no tocar las tablas de la aplicación.
5. Cómo el sistema se recupera solo de errores: timeouts, sesión vencida, caídas.
6. Cómo se asegura que toque la cuenta correcta, que dos corridas no se pisen y que no se filtre la sesión.
7. Las decisiones de diseño, sus alternativas y por qué se eligió cada una (incluida la revisión de código).
8. Qué falta y qué mejorarías (para las preguntas del tribunal).

Repositorio: github.com/SantiGarnier/automatizacion-traxion
Python 3.12 · Playwright · PostgreSQL 18 · octubre de 2026

Índice

1. El bot en 30 segundos . . . 3
2. Contexto y problema . . . 3
3. Arquitectura general . . . 4
4. Mapa del código . . . 5
5. Recorrido de punta a punta . . . 5
6. La plataforma Traxion por dentro . . . 6
7. Concepto clave 1: esperar por evento, no por tiempo . . . 7
8. Concepto clave 2: robustez del lote . . . 8
9. Concepto clave 3: diseño de la base de datos . . . 9
10. Concepto clave 4: el Boss y el Worker . . . 11
11. Seguridad y buenas prácticas . . . 13
12. Configuración . . . 13
13. Cómo se probó . . . 14
14. Guion sugerido para la exposición (12 minutos) . . . 15
15. Preguntas probables del tribunal . . . 15
16. Limitaciones y pendientes . . . 17
17. Glosario . . . 18
18. Autoevaluación . . . 18

Concepto clave
Cómo usar esta guía: leé las secciones 1 a 4 para entender el todo, después las 5 a 9 (los cuatro conceptos técnicos que más preguntan). La sección 14 es el guion para la exposición y la 15 las preguntas probables con sus respuestas. Al final hay un glosario y una autoevaluación.

1. El bot en 30 segundos

> **Para decir en la exposición**
> "En TRASA, cada vez que un comercio de Pago24 Red vincula su cuenta con Traxion, alguien tenía que entrar a la plataforma, buscar la cuenta y poner el Monto crédito en 1. Hicimos un bot en Python que lo hace solo: la app le avisa por HTTP, el bot lee de la base las cuentas pendientes, opera la web de Traxion como lo haría una persona y deja el resultado de cada cuenta en la base, con éxito o con el motivo del error."

Esa es la idea a transmitir en la apertura. Tres palabras clave para retener: **disparo por evento** (no se queda consultando), **automatización web robusta** (espera eventos, no tiempos) y **trazabilidad** (cada cuenta termina con un estado en la base).

Los números de la prueba de punta a punta

Qué	Resultado (9/10/2026, entorno de testing, modo prueba)
Cuentas procesadas	13 (todas las vinculadas a Traxion en la base de stage restaurada)
Tiempo total	≈ 2,5 minutos (12:19:40 → 12:22:11), unos 10 segundos por cuenta
Resultado	11 PRUEBA (habrían quedado en 1) · 2 SIN_CAMBIOS (ya tenían 1) · 0 errores · 0 no encontradas
Hallazgo	Una cuenta (pruebasapp, 55490) tenía 50.000 de crédito. Se decidió: toda cuenta que no tenga 1 pasa a 1, y el valor anterior queda registrado en `monto_anterior`
Revisión de código	Fede revisó el proyecto y marcó 5 puntos (permisos, validaciones, monto fijo, concurrencia, sesión y TLS). Todos aplicados y probados el 14/10/2026 (sección 16).

2. Contexto y problema

Los actores

Actor	Qué es / qué hace
TRASA	La empresa. Gestiona una red de puntos de venta (kioscos, comercios) que cobran y recargan.
Traxion / Pago24	La plataforma web de gestión (testing.telerecargas.com.ar). Ahí se configuran las cuentas comerciales.
Pago24 Red (p24red)	La aplicación donde los comercios se registran. Su base PostgreSQL guarda qué usuario está vinculado a Traxion.
Santiago	Desarrolló el bot.
Fede (Federico Hanson)	Dio las credenciales y la base, definió el contrato de la tabla y el esquema Boss/Worker; arma el servidor.

El problema

Cuando se da de alta un punto de venta, su cuenta en Traxion tiene que quedar con **Monto crédito = 1**. Hacerlo a mano implica: entrar a Traxion, ir a GESTORES → Cuentas, buscar por usuario, abrir la edición con el lápiz naranja de la columna "M", cambiar el campo y aceptar. Es repetitivo, propenso a errores y no escala.

Por qué un bot que usa la web (y no una API)

Traxion no expone una API para esta operación: la única interfaz es la web. Por eso el bot es una **automatización de interfaz** (también llamada RPA o *scraping* activo): un navegador real controlado por código que hace los mismos clicks que haría una persona. Si mañana Traxion ofreciera una API, sería mejor usarla: es más rápida y no depende del diseño de las pantallas.

> **Ojo: posible pregunta**
> "¿Por qué no usaron requests + BeautifulSoup, que es más liviano?" → Porque Traxion es una app **Seaside**: cada click genera URLs nuevas (`_s` de sesión y `_k` de continuación) y la página se arma con JavaScript y AJAX. Reproducir eso a mano con requests sería frágil; un navegador real (Playwright) ejecuta el JavaScript igual que el usuario.

3. Arquitectura general

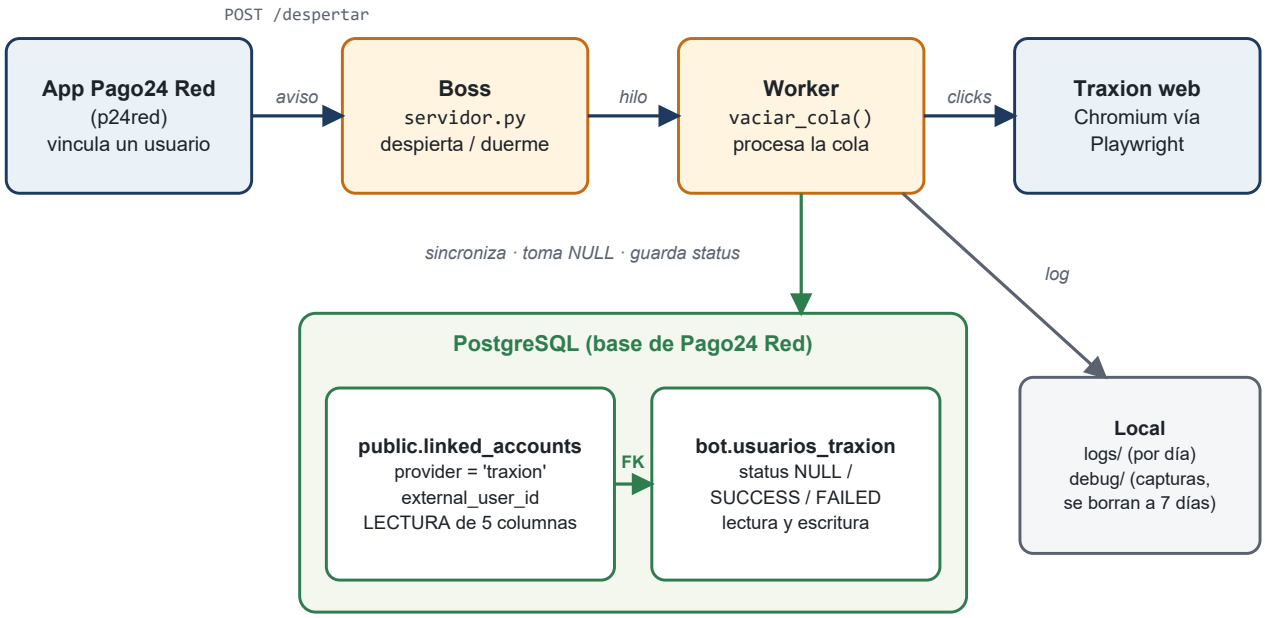


Figura 1. Componentes del sistema y cómo se comunican.

Leé la figura de izquierda a derecha:

- La **app Pago24 Red** vincula un usuario y le avisa al bot con `POST /despertar` (con un token en el header).
- El **Boss** (`servidor.py`) recibe el aviso. Si el Worker duerme, lo despierta en un hilo aparte y responde enseguida.
- El **Worker** sincroniza la base, toma las cuentas pendientes y por cada una maneja un **Chromium** con Playwright contra la web de Traxion.
- El resultado de cada cuenta se guarda en **PostgreSQL** (`bot.usuarios_traxion`), la única fuente de verdad. El detalle de cada intento queda además en el log del día; ante un error se guarda una captura en `debug/`.
- Cuando no quedan pendientes, cierra la sesión en Traxion y el Boss lo manda a dormir: no consume nada hasta el próximo aviso.

Tecnologías y por qué cada una

Tecnología	Para qué	Por qué esta y no otra
Python 3.12	Lenguaje del bot	Ecosistema maduro para automatización y bases de datos; rápido de escribir y leer.
Playwright (Chromium)	Manejar el navegador	Espera automática por estado del DOM, API sincrónica simple, modo headless para servidor. Más moderno que Selenium.
PostgreSQL + psycopg 3	Leer cuentas y guardar resultados	Es la base que ya usa Pago24 Red. psycopg es el driver estándar; consultas parametrizadas.
http.server (librería estándar)	Servidor Boss/Worker	Son 3 endpoints: un framework (FastAPI, Flask) sería más dependencias sin beneficio.
python-dotenv	Configuración	Credenciales en un archivo `.env` fuera del código y fuera de git.

4. Mapa del código

Archivo	Líneas	Responsabilidad
`traxion_bot.py`	823	El bot: login y logout, navegación, búsqueda, validaciones, edición, manejo de errores, re-login, registro, CLI y `vaciar_cola()`.
`servidor.py`	192	Servidor HTTP con las clases `Boss` y `Worker` (endpoints `/despertar`, `/estado`, `/salud`).
`fuente_db.py`	255	Acceso a PostgreSQL: singleton `BaseDatos` (con candado y TLS), `sincronizar()`, `pendientes()`, `reclamar()`, `actualizar()`.
`config.py`	74	Lee todas las variables del `.env` (credenciales, base, timeouts, token, debug).
`sql/01_bot_usuarios_traxion.sql`	95	Crea (o actualiza) el esquema `bot`: tabla, índices, vista y primera sincronización.
`sql/02_usuario_bot.sql`	27	Usuario de base `traxion_bot` con permisos mínimos (5 columnas, sin `users`).
`tests/` (6 archivos)	1.048	Orquestación, base de datos y permisos, concurrencia con procesos reales, servidor, seguridad y validaciones con navegador.
`README.md`, `.env.example`	—	Instalación y uso; plantilla de configuración sin valores reales.

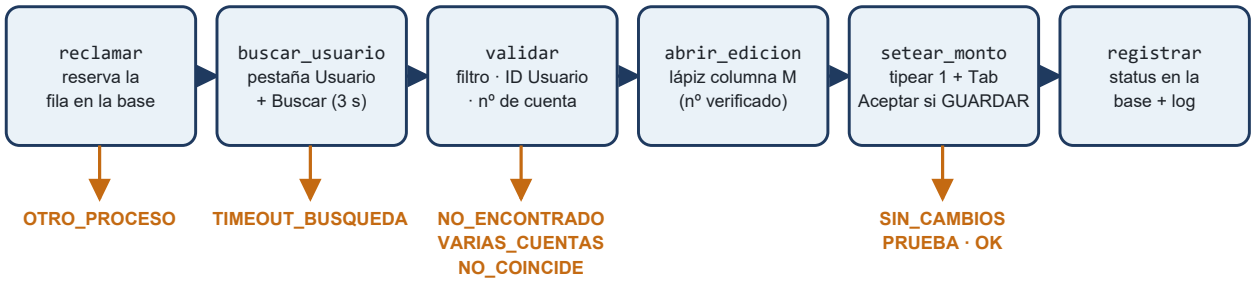
> **Dato concreto**
> Unas 1.350 líneas de Python de producción y unas 1.050 de tests: hay casi tanto código que prueba como código que hace. El archivo más grande es `traxion_bot.py` porque concentra la interacción con la web, que es la parte más delicada.

5. Recorrido de punta a punta

Esta es la historia completa de **una cuenta**, desde que se vincula hasta que queda resuelta. Si en la exposición te piden "contá cómo funciona", contá esto en orden.

1. **Se vincula un comercio.** La app Pago24 Red inserta una fila en `public.linked_accounts` con `provider = 'traxion'` y el usuario de Traxion en `external_user_id` (por ejemplo `cib7kb9v3scb2sltq6yx6cf17`).
2. **La app avisa.** Hace `POST /despertar` al servidor del bot con el header `X-Token`.

3. **El Boss despierta al Worker.** Valida el token, marca `despierto = True`, lanza un hilo y responde `202`.
4. **Sincronización incremental.** El Worker copia a `bot.usuarios_traxion` las cuentas nuevas (las creadas después de la última importada). Entran con `status` en `NULL` = pendiente.
5. **Candado y cola.** El Worker toma el candado de PostgreSQL (si otra corrida trabaja, sale) y toma las filas con `status IS NULL` cuyo vínculo sigue `active`.
6. **Login en Traxion.** Abre Chromium, completa usuario y clave del `.env`, aprieta Ingresar y verifica que aparezca el menú GESTORES.
7. **Por cada cuenta:** la **reserva** en la base → GESTORES → Cuentas → pestaña Usuario → escribe el usuario → Buscar → **valida** (filtro exacto, "ID Usuario" en la pantalla de usuarios, número de cuenta en el formulario) → lápiz de la columna M → campo Monto crédito → tipea 1 → Aceptar (si `GUARDAR=true`).
8. **Registro.** Traduce el resultado a `status` (`SUCCESS`, `FAILED` o sigue `NULL` para reintentar), lo guarda en la base por `linked_account_id` y libera la reserva; además lo anota en el log.
9. **¿Quedan pendientes?** Re-sincroniza (por si entró alguna cuenta mientras trabajaba) y repite. Si no queda ninguna, cierra la sesión en Traxion y el Boss lo manda a dormir.



Todas las salidas (menos OTRO_PROCESO, que es de otra corrida) terminan en registrar().

Figura 2. Los pasos por cuenta dentro del Worker y sus salidas posibles.

6. La plataforma Traxion por dentro

La mitad del trabajo de un bot web es entender la página. Antes de escribir selectores se inspeccionó el HTML real con la sesión iniciada. Estas son las particularidades que condicionan el código:

Particularidad	Consecuencia en el bot
App **Seaside**: URLs con `_s` (sesión) y `_k` (continuación) que cambian en cada click.	No se puede ir directo a una pantalla interna por URL. El bot siempre navega con el menú, como una persona.
Los `name` de los campos son números dinámicos (`name="331"`).	Los campos se ubican por su **etiqueta** ("Usuario", "Monto credito") con XPath, nunca por `name`.
Ajaxifier: cada click hace un AJAX y la respuesta **reemplaza todo el** `<body>`.	Se aprovecha para esperar por evento: ver sección 7.
Hay **dos** links "Cuentas": uno en PROVEEDORES y otro en GESTORES.	El selector busca "Cuentas" **dentro** del ítem GESTORES. Un selector ingenuo tomaba el equivocado.
El submenú GESTORES se despliega y se pliega con el mismo click.	Solo se clickea si "Cuentas" no está visible (desde el segundo usuario ya está abierto).
Las pestañas de filtros son **jQuery UI** (`li[role=tab]`) y **cargan su contenido aparte**: después de Buscar, "Usuario" aparece seleccionada pero vacía un instante.	Se espera que quede seleccionada, sin clase de carga y con su campo adentro; si no carga sola, se fuerza (General → Usuario).
La búsqueda por usuario es **exacta y distingue mayúsculas** (probado a mano).	Una búsqueda no puede traer la cuenta de otro usuario "parecido".

Particularidad	Consecuencia en el bot
El ícono de la columna "Usuarios" abre "Usuarios de la cuenta: <número>" con la columna **"ID Usuario"**.	Se usa para verificar que el usuario buscado esté en esa cuenta. Esa pantalla muestra datos personales: el bot solo lee el título y "ID Usuario".
Cerrar el navegador no cierra la sesión: queda viva del lado de Traxion.	Al terminar, el bot sale: menú del usuario → "Salir de plataforma" → "Salir".
El lápiz naranja de la columna M está en `<span title="Modificar el registro">`.	Se busca por ese título, más firme que contar columnas.
"Monto crédito" usa **autoNumeric** (formatea al tipear y al salir del campo).	Se tipea tecla por tecla (`press_sequentially`) y se sale con **Tab**.
Enter dispara un botón oculto "Default" del formulario.	El bot **nunca** usa Enter; aprieta "Aceptar" explícitamente.
Puede aparecer un **reCAPTCHA** en el login.	El bot se detiene y lo informa. **No** intenta saltearlo.

> **Para decir en la exposición**
> "Lo más difícil no fue programar los clicks, sino entender la página: es una app Seaside donde todo se reemplaza por AJAX, los campos no tienen nombres fijos y hasta hay dos menús con el mismo nombre. Por eso cada selector se validó contra el HTML real antes de usarlo."

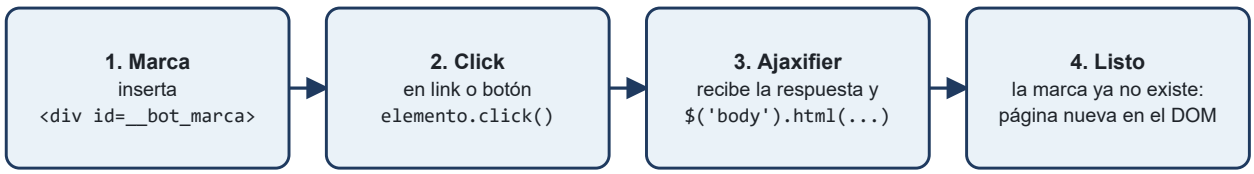
7. Concepto clave 1: esperar por evento, no por tiempo

El problema

La primera versión esperaba después de cada click a que la red quedara quieta (`networkidle`) con un máximo de 5 segundos, y seguía igual si no pasaba. Resultado: **"un día anda y al otro no"**. Si Traxion tardaba 6 segundos, el bot leía la pantalla vieja. Un `sleep(5)` tiene el mismo defecto: o espera de más (lento) o de menos (falla).

La solución: la técnica de la marca

Leyendo el JavaScript de la plataforma (`ajaxifier_v4.7.7.js`) se vio que, al llegar la respuesta de cada click, hace `$("body").html(...)`: reemplaza todo el contenido de la página. Entonces:



`wait_for_function("no existe __bot_marca")` — tope: 3 s en la búsqueda, 60 s en el resto

Si se cumple el tope sin respuesta: SinRespuesta / TimeoutBusqueda. Si la página nueva es el login: SesionVencida.

Figura 3. Cómo funciona click_y_esperar().

```
def click_y_esperar(page, elemento, accion, timeout=None, error_cls=SinRespuesta):
    page.evaluate("id => { const d = document.createElement('div'); d.id = id;"
                  " d.hidden = true; document.body.appendChild(d); }", "__bot_marca")
    elemento.click()
    page.wait_for_function("() => !document.getElementById('__bot_marca')", timeout=timeout)
    # ... verifica HTTP >= 400, detecta pantalla de login (SesionVencida)
```

- Si Traxion responde en 200 ms, el bot sigue a los 200 ms; si tarda 20 s, espera 20 s. **Nunca adivina.**
- Funciona igual si en vez de AJAX hubiera una navegación completa: la marca también desaparece.

- El timeout pasa a ser un **tope de seguridad**, no una espera: 60 s en general, **3 s** en la búsqueda.

> **Concepto clave**
> Un tiempo de espera fijo es una **suposición** sobre la velocidad del servidor. Esperar un evento es una **observación**. Por eso el bot dejó de ser intermitente. Pasos que no van al servidor (desplegar el menú, cambiar de pestaña) también esperan un estado concreto del HTML, no un tiempo.

> **Ojo: posible pregunta**
> "¿Y si el sitio cambia su JavaScript?" → La técnica depende de que la página reemplace el `<body>`. Si eso cambiara, `click_y_esperar` vencería por el tope y el bot lo informaría como error con captura de pantalla; no quedaría colgado ni haría algo incorrecto en silencio.

8. Concepto clave 2: robustez del lote

Un lote de cientos de cuentas no puede cortarse por una falla. La regla de diseño es: **cada cuenta termina con un resultado registrado y el bot sigue con la siguiente**.

Errores tipados

Cada falla es una clase que hereda de `BotError` y lleva su propio estado. Así el código que registra no necesita adivinar qué pasó:

```
class BotError(Exception):            estado = "ERROR"
class SinRespuesta(BotError):         # la plataforma no reemplazó la página a tiempo
class TimeoutBusqueda(SinRespuesta):  estado = "TIMEOUT_BUSQUEDA"
class NoEncontrado(BotError):         estado = "NO_ENCONTRADO"    # "Datos: 0"
class VariasCuentas(BotError):        estado = "VARIAS_CUENTAS"
class NoCoincide(BotError):           estado = "NO_COINCIDE"      # la cuenta no es la del usuario
class SesionVencida(BotError):        # apareció el login: re-loguear y reintentar
```

Validaciones antes de tocar una cuenta

Antes de cambiar nada, el bot exige tres pruebas de que la cuenta abierta es la del usuario buscado. Si alguna no coincide, la cuenta queda `FAILED` con resultado `NO_COINCIDE` y el detalle de qué esperaba y qué encontró:

1. **La grilla es de esta búsqueda:** el filtro "Usuario" de la página nueva tiene que decir exactamente el usuario buscado.
2. **El usuario está en esa cuenta:** en la pantalla de usuarios de la cuenta, el usuario tiene que aparecer en "ID Usuario" con el texto exacto, y el título tiene que ser de esa misma cuenta.
3. **El formulario es de esa cuenta:** el "Número de Cuenta" del formulario tiene que ser el de la grilla.

> **Concepto clave**
> Lección de la primera prueba real (14/10): el bot leyó el filtro antes de que la pestaña terminara de cargar, no encontró el campo y marcó la cuenta como `NO_COINCIDE`, una falla **definitiva**. El error no era de la cuenta sino del bot. Se corrigieron dos cosas: esperar a que la pestaña cargue, y distinguir **"no pude leer"** (`ERROR`, se reintenta) de **"leí otro valor"** (`NO_COINCIDE`). Una validación nunca debe convertir una falla del bot en una conclusión sobre los datos.

Los mecanismos de recuperación

Situación	Qué hace el bot
La búsqueda queda "cargando" más de 3 s	Registra `TIMEOUT_BUSQUEDA`, sigue con el siguiente y al final del lote la reintenta una vez.
Aparece la pantalla de login a mitad del lote	`SesionVencida`: vuelve a entrar y reintenta **esa misma** cuenta, sin contarla como error.
Un error deja la página en mal estado	Marca la sesión como **sucia**: antes de la siguiente cuenta cierra la pestaña y se re-loguea. Evita que una respuesta tardía se mezcle con otra cuenta.
No puede volver a loguearse	Las cuentas restantes quedan como `NO_PROCESADO` (no se pierden: siguen pendientes).
Cualquier error	Guarda una captura de pantalla en `debug/` (sin HTML, para no dejar la sesión en disco).
Otra corrida tiene la cuenta reservada	La saltea (`OTRO_PROCESO`): no la toca ni la registra.
Falla la escritura en la base	No corta el lote. La cuenta sigue pendiente y se reprocesa; como el bot es idempotente, da `SIN_CAMBIOS`.

> **Concepto clave**
> **Idempotencia:** procesar dos veces la misma cuenta no causa daño. Si ya tiene Monto crédito = 1, el bot no toca nada y registra `SIN_CAMBIOS`. Esa propiedad es la que permite reintentar con tranquilidad ante cualquier duda.

9. Concepto clave 3: diseño de la base de datos

Regla de oro: no tocar las tablas de la aplicación

El bot nunca modifica las tablas de la app. De `public.linked_accounts` lee **solo 5 columnas** (`id`, `provider`, `status`, `external_user_id`, `created_at`) y **no lee** `public.users` (emails, CUIT, contraseñas). Todo lo que escribe va a un esquema propio, `bot`. El usuario de base que crea `sql/02_usuario_bot.sql` tiene exactamente esos permisos: si intentara leer otra columna, `users` o la vista de consulta, **la propia base lo rechaza** (hay un test que lo comprueba).

> **Ojo: posible pregunta**
> "¿Por qué no le dieron acceso a la vista `bot.v_usuarios_traxion`?" → Porque la vista hace join con `public.users`, y en PostgreSQL una vista se ejecuta con los permisos de su **dueño**. Si el bot pudiera leerla, tendría una puerta trasera a `users`. La vista es para consulta humana (DBeaver).

La tabla bot.usuarios_traxion

Columna	Para qué
`linked_account_id`	Clave primaria y **FK** a `linked_accounts.id` (`ON DELETE CASCADE`). Todo se reserva y se guarda por esta clave.
`usuario_traxion`	El `external_user_id`: lo que se busca en Traxion. **No es único**: dos vínculos pueden apuntar al mismo usuario.
`origen_creado_en`	El `created_at` original: la **marca de agua** de la sincronización.
`status` / `status_message`	`NULL` pendiente · `SUCCESS` · `FAILED` + motivo. Un `CHECK` impide otros valores.
`resultado`, `intentos`	Detalle del bot (OK, NO_ENCONTRADO…) y fallas pasajeras acumuladas.
`cuenta_numero`, `monto_anterior`, `procesado_en`	Trazabilidad: qué cuenta se tocó, qué valor tenía antes y cuándo.
`reclamado_en`, `reclamado_por`	Reserva mientras una corrida la procesa. Se libera al guardar y vence sola a los 30 min.

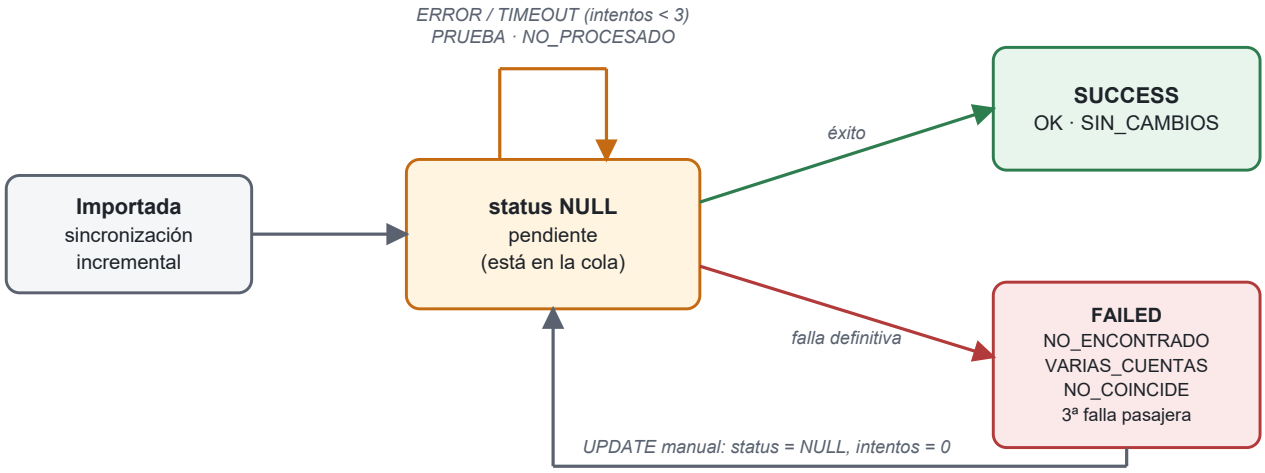


Figura 4. Ciclo de vida de una cuenta en bot.usuarios_traxion.

Tres decisiones para no hacer full scans (pedido explícito de Fede)

- **Sincronización incremental con marca de agua.** Solo trae las cuentas con `created_at` posterior a la última importada, menos un margen de 60 minutos por transacciones que confirman tarde. Con `ON CONFLICT DO NOTHING`, las repetidas se ignoran y **no se reescribe ninguna fila existente**.
- **Índice parcial** `WHERE status IS NULL`: contiene solo las pendientes. Con el tiempo casi todas las filas estarán resueltas, pero la cola sigue siendo chica y rápida. Un test verifica con `EXPLAIN` que la consulta lo usa.
- **Estado del vínculo en vivo por la FK.** No se copia `active`/`inactive`: se lee de `linked_accounts` por clave primaria. Si la app desactiva una cuenta, sale de la cola sin re-sincronizar.

```
INSERT INTO bot.usuarios_traxion (linked_account_id, usuario_traxion, origen_creado_en)
SELECT la.id, la.external_user_id, la.created_at
FROM public.linked_accounts la
WHERE la.provider = 'traxion' AND la.external_user_id IS NOT NULL
  AND la.created_at >= COALESCE((SELECT MAX(origen_creado_en) FROM bot.usuarios_traxion), '-infinity')
                       - make_interval(mins => %(margen_min)s)
ON CONFLICT (linked_account_id) DO NOTHING
```

Fallas pasajeras vs. definitivas

Si una cuenta falla porque Traxion tardó o se cayó la sesión, marcarla `FAILED` sería injusto: un problema momentáneo la dejaría marcada para siempre. Por eso las fallas **pasajeras** (`ERROR`, `TIMEOUT_BUSQUEDA`) dejan `status` en `NULL` con el motivo en `status_message`, y recién al **tercer** intento pasan a `FAILED`. Las **definitivas** (`NO_ENCONTRADO`, `VARIAS_CUENTAS`, `NO_COINCIDE`) van directo a `FAILED`, porque reintentar no cambia nada.

Varias corridas a la vez: candado y reserva

El `threading.Lock` del Boss solo coordina los hilos de **un** proceso. Si además corre un cron (u otro servidor), son dos procesos: los dos leerían las mismas cuentas pendientes y podrían editar la misma cuenta a la vez. Fede lo marcó en la revisión. Se resolvió en dos capas:

- **Candado por corrida** (`pg_try_advisory_lock`): al arrancar, cada corrida lo pide a PostgreSQL. Si otra lo tiene, sale enseguida sin tocar nada. Está atado a la conexión: si el proceso muere, la base lo libera sola. Si la conexión se cae, al reconectar lo vuelve a pedir y, si otra corrida lo tomó, corta.
- **Reserva por fila:** antes de abrir una cuenta en Traxion, un `UPDATE … RETURNING` atómico la marca como reservada por este proceso. Si dos procesos la piden a la vez, PostgreSQL bloquea la fila y solo uno la recibe. La reserva vence sola si el proceso murió con la cuenta tomada.
- **Todo por `linked_account_id`:** antes el resultado se guardaba por usuario de Traxion, y dos vínculos con el mismo usuario se pisaban. Ahora cada vínculo es un trabajo con su propio resultado.

> **Dato concreto**
> Probado con **procesos reales** del sistema operativo: con servidor y cron a la vez trabajó uno solo y procesó las 8 cuentas una vez cada una; con dos procesos peleando las mismas filas sin candado, se repartieron 4 y 4 sin ninguna repetida.

Conexión: singleton y sin ORM

- `BaseDatos` es un **singleton**: `BaseDatos()` devuelve siempre la misma instancia, con una única conexión. Se conecta recién la primera vez que se usa (*lazy*) y se reconecta sola si la conexión se cayó.
- `autocommit=True`: cada resultado queda guardado al instante, aunque el bot se corte después.
- **TLS obligatorio fuera de localhost:** por el canal viajan la clave y los ids de cuenta. Con una base remota, el bot usa `sslmode=verify-full` (cifra y comprueba el certificado del servidor) y **se niega a arrancar** con un modo que permita ir sin cifrar.
- **Sin ORM.** Fede lo sugirió para evitar inyección SQL, pero las consultas ya van **parametrizadas** (`%(nombre)s`), que es lo que realmente la previene. Son tres consultas muy de PostgreSQL (`ON CONFLICT`, índice parcial, `CASE`): un ORM agregaría otra sintaxis sin portabilidad real. Fede coincidió: "una bazooka para matar un mosquito".

10. Concepto clave 4: el Boss y el Worker

Por qué no un cron que consulte cada X minutos

Un cron (*polling*) pregunta a la base todo el tiempo aunque no haya nada, y además agrega demora: una cuenta vinculada justo después de una consulta espera hasta la siguiente. El esquema **Boss/Worker** es **dirigido por eventos**: el bot duerme sin consumir nada y la app lo despierta exactamente cuando hay trabajo. Fue el flujo que Fede dibujó en la reunión.

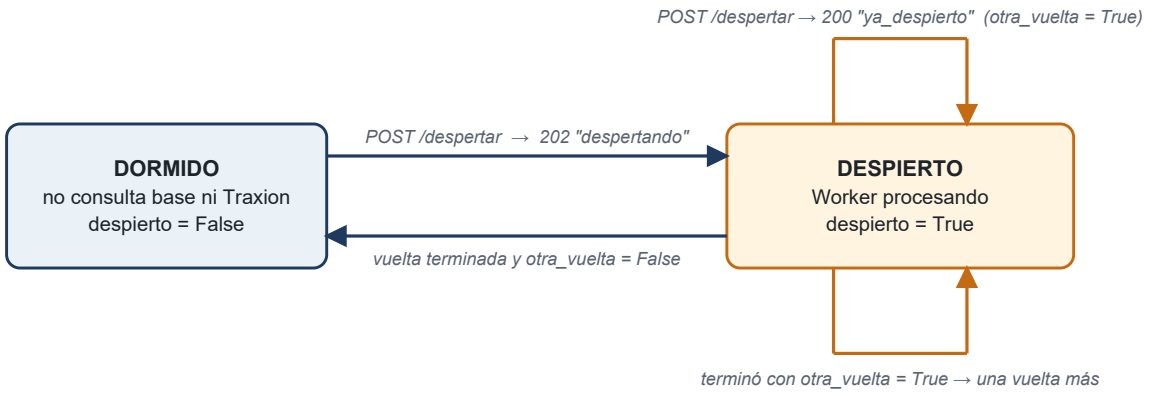


Figura 5. Máquina de estados del Boss.

Las reglas

- **Un solo Worker a la vez.** Un `threading.Lock` protege el estado. Si llega un aviso mientras trabaja, no arranca otro: anota `otra_vuelta = True` y responde `200 "ya_despierto"`.
- **Una vuelta más, no N.** Al terminar, si hubo avisos en el medio, da exactamente una vuelta extra para tomar lo nuevo (diez avisos seguidos no generan diez vueltas).
- **Cada cuenta, una vez por despertar.** `vaciar_cola()` recuerda qué cuentas ya vio. Sin esto, con Traxion caído, una cuenta con falla pasajera volvería a la cola, se reintentaría en loop y el Worker **nunca se dormiría**.
- **El Boss no se cae.** Si una vuelta falla (por ejemplo, la base no responde), lo registra en `/estado` con `"ok": false` y vuelve a dormir; el próximo aviso lo intenta de nuevo.
- **Entre procesos manda el candado de la base.** El `Lock` del Boss no ve a otros procesos; por eso el Worker toma además el candado de PostgreSQL al empezar cada vuelta (sección 9). Así es seguro sumar un cron como red de seguridad.

Los endpoints

Llamada	Respuesta	Uso
`POST /despertar`	`202 despertando` o `200 ya_despierto`	Lo llama la app al vincular un usuario.
`GET /estado`	`dormido`/`despierto` + última vuelta (resultados o error)	Monitoreo y diagnóstico.
`GET /salud`	`{"ok": true}`	Chequeo de vida. El único sin token.

> **Concepto clave**
> Seguridad del endpoint: todas las llamadas (menos `/salud`) exigen el header `X-Token`. Se compara con `hmac.compare_digest`, que tarda lo mismo acierte o no, para que no se pueda adivinar el token midiendo tiempos (*timing attack*). El servidor no arranca si el token tiene menos de 16 caracteres ni si no puede conectarse a la base: los errores de configuración aparecen al iniciar, no al primer aviso.

11. Seguridad y buenas prácticas

Riesgo	Cómo se mitiga
Filtrar credenciales	Todo secreto vive en `.env`, excluido de git. `.env.example` es una plantilla **vacía**. Antes de cada commit se buscó que ningún valor del `.env` apareciera en los archivos a subir (y se detectó y corrigió una clave copiada por error en la plantilla).
Romper datos de la app o leer de más	El bot solo lee 5 columnas de `linked_accounts` y nada de `users`. Usuario de base con permisos mínimos por columna (principio de **menor privilegio**); la base rechaza cualquier otra lectura.
Modificar la cuenta equivocada	Tres validaciones antes de Aceptar (filtro, "ID Usuario", número de cuenta). Ante la duda, `NO_COINCIDE` y no toca nada.
Dos corridas editando lo mismo	Candado de PostgreSQL por corrida + reserva atómica de cada fila.
Robar la sesión de Traxion	Logout al terminar cada lote; `_s`/`_k` tapados en logs y mensajes; debug sin HTML por defecto; capturas borradas a los 7 días.
Interceptar la conexión a la base	TLS obligatorio (`verify-full`) con bases remotas; el bot no arranca sin cifrado.
Un monto equivocado aplicado a todo	El valor 1 es una constante en el código, no configurable.
Guardar algo indebido	**Modo prueba por defecto** (`GUARDAR=false`): carga el valor pero no aprieta Aceptar. El guardado real requiere activarlo explícitamente.
Valores inválidos en la base	`CHECK (status IN ('SUCCESS','FAILED'))`.
Inyección SQL	Consultas siempre parametrizadas; nunca se concatenan valores en el SQL.
Acceso no autorizado al servidor	Token obligatorio, comparación en tiempo constante; por defecto escucha solo en `127.0.0.1`; si se expone, detrás de HTTPS.
Saltar controles de la plataforma	Ante un captcha el bot se detiene: no lo evade.
Datos personales	El dump de stage y los logs (que mencionan cuentas) no se suben al repositorio.

12. Configuración

Todo se configura en el `.env`. Las variables más importantes:

Variable	Default	Qué controla
`TRAXION_URL`, `TRAXION_USER`, `TRAXION_PASS`	testing	Entorno y credenciales de Traxion. Pasar a productivo = cambiar la URL.
`GUARDAR`	`false`	Si aprieta Aceptar. En `false` es modo prueba.
`HEADLESS`	`false`	Sin ventana de navegador (en el servidor, `true`).
`TIMEOUT_MS` / `TIMEOUT_BUSQUEDA_MS`	60000 / 3000	Topes de seguridad (no esperas fijas).
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS`	—	Conexión a PostgreSQL. Sin `DB_NAME` el bot no usa la base.
`MAX_INTENTOS`	3	Fallas pasajeras antes de `FAILED`.
`SYNC_MARGEN_MIN`	60	Margen hacia atrás de la sincronización incremental.
`RECLAMO_VENCE_MIN`	30	Vencimiento de la reserva de una cuenta si el proceso murió.

Variable	Default	Qué controla
`DB_SSLMODE`, `DB_SSLROOTCERT`	automático	TLS: `verify-full` con bases remotas (no se acepta sin cifrar).
`DEBUG_HTML`, `DEBUG_RETENCION_DIAS`	false, 7	HTML de debug (solo para desarrollar) y días que se guardan las capturas.
`API_HOST`, `API_PORT`, `API_TOKEN`	127.0.0.1, 8080, —	Dónde escucha el Boss y su token.

Formas de ejecutarlo

```
python traxion_bot.py --solo-login          # probar credenciales
python traxion_bot.py --limite 1            # una cuenta pendiente de la base, modo prueba
python traxion_bot.py --usuario X           # una cuenta puntual (no toca la base)
python servidor.py                          # Boss/Worker: queda dormido esperando POST /despertar
curl -X POST -H "X-Token: <token>" http://localhost:8080/despertar
```

13. Cómo se probó

Se probó en capas, de lo más aislado a lo más real, **sin tocar nunca stage ni productivo con datos de prueba**:

Prueba	Qué verifica	Cómo
`tests/test_orquestacion.py`	Re-login por sesión vencida, timeouts, `NO_PROCESADO`, `OTRO_PROCESO`, vínculos con el mismo usuario, monto fijo.	Simula el resultado de cada cuenta: no abre navegador.
`tests/test_fuente_db.py`	Sincronización incremental, FK, índice parcial, reintentos, reserva por fila, candado (con reconexión) y **permisos mínimos** con un usuario real de base.	Contra un PostgreSQL **descartable** (puerto 5544) creado y borrado para la prueba. Se niega a correr si la base no se llama `prueba*`/`test*`.
`tests/test_concurrencia.py`	Servidor + cron a la vez (trabaja uno solo) y dos procesos compitiendo por las filas (ninguna repetida).	Con **procesos reales** del sistema operativo.
`tests/test_servidor.py`	Token, despierto/dormido, una sola vuelta extra, recuperación tras un error.	Con un Worker simulado que el test pausa y libera.
`tests/test_seguridad.py`	`_s`/`_k` tapados, debug sin HTML y con borrado, TLS obligatorio fuera de localhost.	Sin navegador ni base.
`tests/test_validaciones.py`	Las tres validaciones, la pestaña que carga tarde, "no pude leer" ≠ "no coincide", y el logout.	Navegador real contra páginas que imitan a Traxion.
Selectores	Que cada selector encuentre exactamente un elemento.	Inspeccionando el HTML real de Traxion con la sesión iniciada.
Punta a punta	Todo el flujo real: servidor, base local restaurada de stage y Traxion testing.	13 cuentas en modo prueba: 11 PRUEBA, 2 SIN_CAMBIOS, 0 errores. Después de la revisión de Fede: validaciones y logout verificados en una cuenta real.

Para decir en la exposición
"Los tests no dependen de Traxion: simulan el navegador o usan una base descartable. La prueba real se hizo en modo prueba, así que pudimos recorrer 13 cuentas reales sin modificar ninguna."

14. Guion sugerido para la exposición (12 minutos)

Min.	Bloque	Qué decir / mostrar
0–1	Apertura	El bot en 30 segundos (sección 1). El problema: tarea manual repetitiva en una web sin API.
1–3	Arquitectura	Figura 1. Los cuatro componentes y el flujo app → Boss → Worker → Traxion + base.
3–5	Recorrido	La historia de una cuenta (sección 5), apoyado en la Figura 2.
5–7	Desafío técnico 1	Seaside + AJAX y la técnica de la marca (Figura 3). Contar el "un día anda y al otro no" y cómo se resolvió.
7–9	Desafío técnico 2	La base: tabla propia con FK, sincronización incremental, índice parcial y contrato `status` (Figura 4).
9–10	Boss/Worker	Por qué eventos y no cron; la regla "una vez por despertar" (Figura 5).
10–11	Calidad	La revisión de Fede: validaciones antes de Aceptar, candado entre procesos, permisos mínimos, sesión y TLS. Contar la lección de "no pude leer" ≠ "no coincide".
11–12	Cierre	Resultado de la prueba real, el hallazgo de los 50.000 y los próximos pasos (stage, productivo).

> **Para decir en la exposición**
> Cierre sugerido: "El bot ya recorre la base, opera Traxion y deja todo trazado. Pasó por una revisión de código que encontró cinco riesgos reales, desde dos corridas pisándose hasta la sesión quedando en disco, y los cerramos con pruebas para cada uno. Automatizar no es solo hacer clicks: es hacerlo de forma segura y verificable."

15. Preguntas probables del tribunal

1. ¿Por qué no usaron una API de Traxion?
No existe una API para esta operación; la única interfaz es la web. Si existiera, sería preferible: más rápida y sin depender de las pantallas.

2. ¿Por qué Playwright y no requests o Selenium?
Requests no ejecuta JavaScript y Traxion depende de AJAX y URLs de continuación Seaside. Playwright maneja un navegador real, espera estados del DOM y corre headless; es más moderno y estable que Selenium.

3. ¿Qué pasa si cambian el diseño de la pantalla?
Los selectores son por etiquetas y textos estables, no por posiciones. Si algo cambia, el paso falla con un mensaje claro y captura en `debug/`; la cuenta queda pendiente. Es el riesgo inherente de automatizar una UI.

4. ¿Cómo evitan procesar dos veces una cuenta, incluso con dos procesos a la vez?
Tres capas: un candado de PostgreSQL deja trabajar a una sola corrida; cada cuenta se reserva con un `UPDATE` atómico antes de abrirla; y al terminar queda `SUCCESS` o `FAILED`, fuera de la cola. Además el bot es idempotente: si ya tiene 1, registra `SIN_CAMBIOS`.

5. ¿Cómo saben que modificaron la cuenta correcta?
Tres validaciones antes de Aceptar: el filtro de la grilla dice exactamente el usuario, el usuario aparece en "ID Usuario" de esa cuenta, y el número de cuenta del formulario coincide. Además se comprobó que la búsqueda de Traxion es exacta.

6. ¿Qué pasa si el bot se corta a la mitad?
Cada resultado se guarda al instante (`autocommit`). Lo procesado queda guardado; lo que no, sigue en `NULL`. Si quedó una cuenta reservada, la reserva vence a los 30 minutos y el candado se libera solo al morir la conexión.

7. ¿Por qué un solo Worker? ¿Escala?
Una sola sesión de Traxion evita conflictos y alcanza para el volumen actual (~10 s por cuenta). La reserva por fila ya está preparada para varios Workers: si hiciera falta escalar, cada uno tomaría filas distintas.

8. ¿Por qué el monto está fijo en el código?
Porque en la práctica es un ON/OFF. Configurable, un `.env` mal cargado aplicaría un valor equivocado a todas las cuentas; fijo, cambiarlo requiere tocar el código y pasar por revisión.

9. ¿Cómo protegen la sesión de Traxion?
Con los `_s`/`_k` de una página logueada se puede entrar como el operador. Por eso el bot cierra sesión al terminar, tapa esos valores en todo lo que escribe y no guarda el HTML por defecto.

10. ¿Por qué no un cron?
Un cron consulta aunque no haya trabajo y agrega demora. Con el Boss/Worker la app avisa exactamente cuando hay algo y el bot no consume nada mientras duerme. Igual se puede combinar con un cron que llame a `/despertar` como red de seguridad.

11. ¿Por qué una tabla propia y no un flag en linked_accounts?
Para no escribir en tablas de la aplicación: el bot no necesita permisos de escritura sobre ellas y los cambios de esquema de la app y del bot no se pisan. Con FK e índice parcial el rendimiento es el mismo.

12. ¿Qué es un índice parcial y por qué lo usaron?
Un índice que solo incluye las filas que cumplen una condición (`status IS NULL`). La cola se consulta siempre por pendientes, y el índice se mantiene chico aunque la tabla crezca con el historial.

13. ¿Para qué el margen en la marca de agua?
`created_at` toma la hora de inicio de la transacción. Si una transacción confirma tarde, su fila puede tener una hora anterior a la última importada; el margen de 60 minutos la vuelve a mirar. `ON CONFLICT DO NOTHING` evita duplicados.

14. ¿Cómo previenen la inyección SQL sin ORM?
Con consultas parametrizadas: los valores viajan separados del SQL y el driver nunca los interpreta como código. El ORM no agregaba protección adicional.

15. ¿Por qué 3 segundos para la búsqueda?
Para que un usuario que deja la búsqueda "cargando" no frene el lote. No se pierde: queda pendiente y se reintenta. Es configurable (`TIMEOUT_BUSQUEDA_MS`) si Traxion resultara más lento.

16. ¿Qué pasa con el captcha?
Si aparece, el bot se detiene y lo informa. No se intenta evadir: es un control de seguridad de la plataforma.

17. ¿Cómo probaron sin tocar producción?
Tests con navegador simulado y con un PostgreSQL descartable; base local restaurada desde stage; Traxion de testing y modo prueba (`GUARDAR=false`).

18. ¿Qué mejorarías?
Correrlo como servicio con monitoreo de `/salud` y alertas cuando hay `FAILED`; métricas por vuelta; varios Workers si crece el volumen; pasar a una API si Traxion la ofrece.

16. Limitaciones y pendientes

- **Guardado real** (`GUARDAR=true`) todavía no probado contra una cuenta de testing.
- Confirmar si se procesan solo vínculos `active` o también otros estados.
- **Stage:** crear el esquema `bot` y el usuario `traxion_bot`, armar el servidor como servicio, generar un token propio.
- Definir quién llama a `/despertar` (la app, un cron de respaldo o ambos) y desde dónde (red interna, HTTPS).
- Limitación estructural: depende de la interfaz web de Traxion; un rediseño de pantallas requiere ajustar selectores.

Cómo evolucionó el proyecto (útil para contar el proceso)

Etapa	Qué se agregó
1	Login y flujo completo con selectores validados contra el HTML real.
2	Esperas por evento (técnica de la marca) en lugar de tiempos fijos.
3	Listas de usuarios, `NO_ENCONTRADO`, tope de 3 s, re-login y reintentos; README y repositorio en GitHub.
4	PostgreSQL: lectura de `linked_accounts`, esquema `bot`, restore del dump de stage.
5	Feedback de Fede: FK, sincronización incremental, contrato `status`, índice parcial, singleton (sin ORM).
6	Boss/Worker por HTTP y prueba de punta a punta con 13 cuentas reales en modo prueba.
7	Revisión de código de Fede: permisos por columna, validaciones antes de Aceptar, monto fijo, candado y reserva entre procesos, logout y sesión tapada, TLS, y se sacó SQLite. 6 suites de tests.

17. Glosario

Término	Definición breve
RPA / automatización de UI	Software que opera una interfaz gráfica como lo haría una persona.
Playwright	Biblioteca para controlar navegadores (Chromium, Firefox, WebKit) desde código.
Headless	Navegador sin ventana visible; se usa en servidores.
DOM	La estructura en árbol de la página que el navegador construye a partir del HTML.
Selector / XPath	Expresión que identifica un elemento del DOM (por CSS o por ruta XPath).
AJAX	Pedido al servidor desde JavaScript sin recargar la página completa.
Seaside / continuación	Framework web (Smalltalk) que guarda el estado de la navegación en URLs con claves `_s` y `_k`.
Polling vs. eventos	Consultar periódicamente si hay algo nuevo vs. reaccionar cuando alguien avisa.
Idempotencia	Ejecutar una operación varias veces produce el mismo resultado que una sola vez.
Singleton	Patrón que garantiza una única instancia de una clase (acá, una única conexión a la base).
FK (clave foránea)	Restricción que obliga a que un valor exista en otra tabla; permite joinear e integridad referencial.
Índice parcial	Índice que solo contiene las filas que cumplen una condición.
Marca de agua (watermark)	Último valor procesado (acá, `created_at`) que marca desde dónde seguir.
Autocommit	Cada sentencia se confirma sola, sin esperar a cerrar una transacción.
Inyección SQL	Ataque que mete código SQL a través de un dato; se previene con consultas parametrizadas.
Timing attack	Deducir un secreto midiendo cuánto tarda una comparación; se evita con `hmac.compare_digest`.
Menor privilegio	Dar a cada componente solo los permisos que necesita.
Race condition	Error que depende del orden en que ocurren eventos concurrentes; el `Lock` del Boss y el candado de la base lo evitan.
Advisory lock	Candado de PostgreSQL con una clave elegida por la aplicación; acá coordina las corridas entre procesos.
Reserva (claim) atómica	Marcar una fila como tomada con un solo `UPDATE … RETURNING`: si dos procesos la piden, solo uno la recibe.
TLS / sslmode	Cifrado de la conexión. `verify-full` además comprueba que el servidor sea el esperado.

18. Autoevaluación

Antes de exponer, deberías poder responder cada punto **sin mirar la guía**:

- Explicar el problema y la solución en 30 segundos.
- Dibujar de memoria la Figura 1 (componentes y flechas).
- Contar la historia de una cuenta en los 9 pasos de la sección 5.
- Explicar por qué un `sleep()` falla y cómo funciona la técnica de la marca.
- Nombrar 3 particularidades de Traxion y cómo las resuelve el bot.
- Explicar qué significan `NULL`, `SUCCESS` y `FAILED` y cuándo una falla se reintenta.
- Explicar la sincronización incremental, el margen y el índice parcial.

- Explicar por qué no hay ORM y cómo se previene igual la inyección SQL.
- Describir la máquina de estados del Boss y por qué cada cuenta se procesa una vez por despertar.
- Nombrar las tres validaciones antes de Aceptar y por qué "no pude leer" no es "no coincide".
- Explicar cómo se evita que dos procesos editen la misma cuenta (candado + reserva).
- Nombrar 4 medidas de seguridad (permisos, sesión, TLS, monto fijo…).
- Contar el resultado de la prueba real y el caso de los 50.000.
- Responder 5 preguntas de la sección 15 elegidas al azar.

Para decir en la exposición

Consejo final: no leas el código en la exposición. Mostrá los diagramas y contá **decisiones**: qué problema apareció, qué alternativas había y por qué elegiste cada una. Eso es lo que distingue una exposición de ingeniería de una demo.